Python Tuples & Membership Testing

PYTHON 3

Developer cheat sheet · quick-glance reference

Blue = tuples Amber = membership testing (in / not in)

1 · Tuple at a glance

- **Ordered** – items keep their position; read them with `t[i]`
- **Immutable** – no add, remove or reassign after creation
- **Allows duplicates** – `(1, 1, 2)` is valid
- **Any types** – mix ints, strs, lists, even other tuples
- **Hashable** – can be a dict key / set item if all its items are hashable
- **The comma makes the tuple**, not the parentheses

2 · Creating tuples

```
t = (1, 2, 3)              # parentheses
t = 1, 2, 3                # parens are optional
one = (5,)                 # trailing comma needed!
oops = (5)                 # int, NOT a tuple
empty = ()                 # or tuple()
tuple([1, 2, 3])           # from any iterable
tuple("abc")               # ('a', 'b', 'c')
tuple(range(3))            # (0, 1, 2)
rec = ("Asha", 28, True) # mixed types are fine
```

3 · Indexing & slicing

```
index:   0   1   2   3   4
value:  10  20  30  40  50
neg:    -5  -4  -3  -2  -1
```

```
t = (10, 20, 30, 40, 50)
t[0]       # 10               first
t[-1]      # 50               last
t[1:4]     # (20, 30, 40)     stop excluded
t[:2]      # (10, 20)
t[3:]      # (40, 50)
t[::2]     # (10, 30, 50)     every 2nd
t[::-1]    # (50, 40, 30, 20, 10)  reversed
t[10]      # IndexError
t[10:]     # ()  slices never raise
```

4 · Immutability

```
t = (1, 2, 3)
t[0] = 99      # TypeError: no item assignment
t.append(4)    # AttributeError
del t[0]       # TypeError
t += (4,)      # OK: builds a NEW tuple, rebinds t
```

```
# a mutable item inside CAN still change
t = ([1, 2], 3)
t[0].append(9)   # t is now ([1, 2, 9], 3)

# "edit" a tuple: list -> change -> tuple
lst = list(t); lst[1] = 0; t = tuple(lst)
```

5 · Methods & built-ins

A tuple has only **two** methods: `count()` and `index()`.

Expression	Result
`t.count(x)`	times x appears
`t.index(x)`	first position of x (ValueError if absent)
`t.index(x, i, j)`	search only inside t[i:j]
`len(t)`	number of items
`min(t) max(t) sum(t)`	smallest, largest, total
`sorted(t)`	new sorted **list** (not a tuple)
`tuple(reversed(t))`	reversed copy, as a tuple
`t1 + t2`	concatenate into a new tuple
`t * 3`	repeat into a new tuple
`(1, 2) < (1, 3)`	True – compared item by item

6 · Packing & unpacking

```
pt = (3, 4)                  # packing
x, y = pt                    # x=3, y=4
a, *rest = (1, 2, 3, 4)      # a=1, rest=[2, 3, 4]
h, *mid, z = (1, 2, 3, 4, 5)  # mid=[2, 3, 4]
_, y = pt                    # _ = ignore a value
(p, (q, r)) = (1, (2, 3))    # nested unpacking
a, b = b, a                  # swap, no temp var!
```

```
def min_max(nums):
    return min(nums), max(nums)   # returns a tuple

lo, hi = min_max([4, 9, 1])       # lo=1, hi=9
for i, ch in enumerate("ab"): ...
for key, val in d.items(): ...
```

***rest is always a list.** Count mismatch → `ValueError`

7 · namedtuple – tuple with labels

```
from collections import namedtuple
Point = namedtuple("Point", "x y")
p = Point(3, 4)
p.x, p[1]          # 3, 4  (by name or index)
p._replace(x=9)    # Point(x=9, y=4)
p._asdict()        # {'x': 3, 'y': 4}
```

Still immutable, still a tuple (`p == (3, 4)` is True). Use `typing.NamedTuple` for a type-hinted class version.

8 · Membership testing: in / not in

```
3 in (1, 2, 3)               # True
5 in (1, 2, 3)               # False
"x" not in ("a", "b")        # True
(1, 2) in [(1, 2), (3, 4)]   # True (tuple as item)

if ext in (".jpg", ".png"):  # typical guard
    ...
if x in t:                   # guard before index()
    pos = t.index(x)
```

- Returns a **bool**; works on any container: tuple, list, set, dict (keys), range, and str (substring).
- Write `x not in t`, not `not x in t` – same result, easier to read.

9 · How in matches (rules & gotchas)

- Each item is checked with `is` **first, then** `==` (identity, then equality).
- **Top level only** – it does not look inside nested tuples:

```
2 in (1, (2, 3))        # False
(2, 3) in (1, (2, 3))   # True
```

- **Whole item vs substring** – tuples match whole items, strings match substrings:

```
"a" in ("abc", "def")   # False
"a" in "abc"            # True
```

- **Equal values match** – `1 == 1.0 == True`, so `True in (1, 2)` is True.
- **NaN quirk** – `nan in (nan,)` is True (same object), but `float("nan") in (float("nan"),)` is False.

10 · Speed: tuple vs set lookup

Container	x in c	Notes
tuple / list	O(n)	scans item by item
set / dict	O(1) avg	hash lookup; items must be hashable

```
ALLOWED = {".jpg", ".png", ".gif"}   # built once
if ext in ALLOWED:                   # O(1) lookup
    ...
```

Small fixed list of literals → a tuple is perfectly fine and readable. Big collection, many lookups → use a set.

11 · in vs count / index / any

```
x in t                  # yes/no, stops at 1st match
t.count(x) > 0          # works, but scans all
t.index(x)              # need the position?
any(s.lower() == "y" for s in t)  # custom rule
```

Use `in` for a yes/no answer, `index()` for the position, and `any()` when matching needs custom logic.

12 · Handy tuple + in patterns

```
# tuple of options in stdlib calls
url.startswith(("http://", "https://"))
name.endswith((".jpg", ".png"))
isinstance(x, (int, float))

# tuples as dict keys (coordinates etc.)
grid = {(0, 0): "origin", (1, 2): "A"}
grid[(1, 2)]        # "A"
(0, 0) in grid      # True (tests keys)

# in inside any() / all()
any(c in "aeiou" for c in word)
all(n in (1, 2, 3) for n in nums)
```

13 · Tuple vs list

	tuple	list
Syntax	(1, 2)	[1, 2]
Mutable	No	Yes
Methods	2 (count, index)	many (append, sort…)
Dict key / set item	Yes*	No
Memory / speed	smaller, a bit faster	more overhead
Best for	fixed records, returns	growing collections

* only if every item inside is hashable.

14 · Common errors

You see	Why / fix
(5) is an int	missing comma → write (5,)
TypeError: …does not support item assignment	tuples are immutable → build a new one
ValueError: too many / not enough values to unpack	names ≠ items; use *rest
ValueError: tuple.index(x): x not in tuple	guard with if x in t
IndexError: tuple index out of range	index must be < len(t)
TypeError: unhashable type: 'list'	list inside a tuple used as dict key / set item

REMEMBER

- The **comma** makes the tuple – `(5,)`, not `(5)`
- Tuple = fixed. List = flexible.
- `in` checks **top-level items** using **==** (identity first).
- Many lookups? Convert to a **set**.
- Swap with a tuple: `a, b = b, a`

SAMPLE PROGRAM

Swapping Temperature

The problem

You are building a temperature converter app. Sometimes, due to incorrect input order, the `min_temp` and `max_temp` values are **swapped**. Use Python tuples to put them back in the correct order.

Variable	Current (wrong)	Expected (after swap)
min_temp	40	25
max_temp	25	40

Solution

```
# Temperature Converter App - fix swapped min/max inputs
min_temp = 40
max_temp = 25

print("Before swap:")
print(f"  min_temp = {min_temp}, max_temp = {max_temp}")

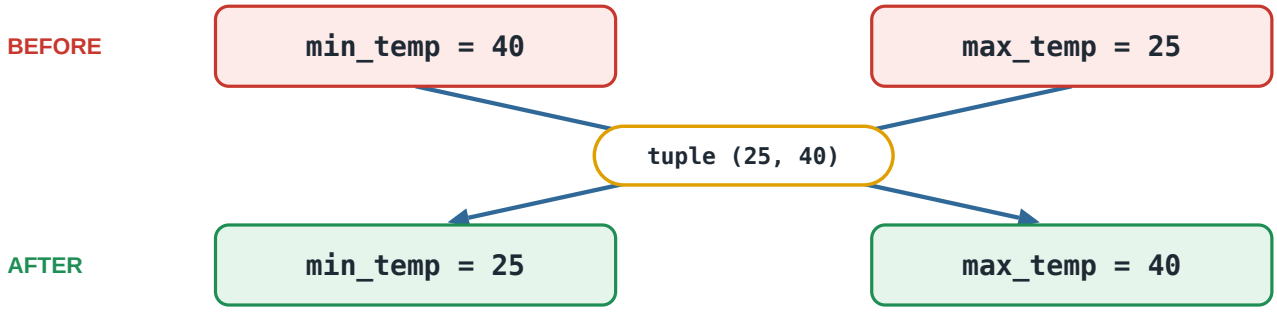
# Tuple packing + unpacking swaps both values in ONE line
min_temp, max_temp = max_temp, min_temp

print("After swap:")
print(f"  min_temp = {min_temp}, max_temp = {max_temp}")
```

```
$ python swap_temps.py
```

```
Before swap:
  min_temp = 40, max_temp = 25
After swap:
  min_temp = 25, max_temp = 40
```

How the swap works



1. **Right side first.** Python evaluates `max_temp, min_temp` and packs the values into a temporary tuple `(25, 40)`.
2. **Then unpack, left to right.** `min_temp` receives 25, then `max_temp` receives 40.
3. **No temp variable needed.** The tuple already holds both old values, so nothing is overwritten too early.

Other ways to write it

Swap only if out of order

```
if min_temp > max_temp:
    min_temp, max_temp = max_temp, min_temp
```

One-liner with sorted()

```
min_temp, max_temp = sorted((min_temp, max_temp))
```

Old way: temporary variable

```
temp = min_temp
min_temp = max_temp
max_temp = temp
```

Membership tie-in: `in` tests equality with the items, not a range. After the swap, `30 in (min_temp, max_temp)` is False. For a range check use `min_temp <= 30 <= max_temp` (True).